Portfolio Data Folder

One folder = one portfolio · relative paths, movable as a whole · convention over hunting

What is it?

Portfolio configs used to point at scattered files with absolute paths — the folder did not survive a move. The data folder is a folder convention: portfolio.json on top, one subfolder per solution with solution.json, arts/ (IssueTimes/CFD/Transitions) and registers/ (the nine registers under standard names), plus snapshots/, raw/ and workflows/. Relative paths in a config resolve against the config file's folder — the folder moves, copies, zips and travels as a whole.

How to use it

```
python -m testdata_generator --scenario portfolio --output demo/ --seed 42
# move demo/ anywhere – then:
python -m portfolio demo/portfolio.json --output report.html
python -m portfolio --delta demo/snapshots/snapshot_prev.json demo/snapshots/snapshot_now.json
```

The demo scenario produces the structure ready-made; for your own portfolios just put config and data into one shared folder. In the GUI, the 'Data sources …' dialog manages the nine register paths with a per-field status; 'Take from data folder …' fills them by convention, and on save, paths inside the config folder are relativised automatically.

The rules

• Generated variety: the demo scenario's registers are fixed story anchors plus a seed-generated base population — scales s/m/l (CLI --scale, GUI picker; default m), anchors identical at every scale
• Relative paths: relative to the config file — one rule everywhere (members, templates, all nine registers)
• Absolute paths: unchanged — existing configs keep working
• Legacy CWD-relative: still works via fallback, with a log hint
• Standard names in registers/ are a convention, not auto-discovery — an empty config field still means 'no register'

Guardrails

The configs stay the single source of truth; the convention only keeps them short and portable. No schema bump, no silent behaviour change. A move-the-folder test in the suite proves portability on every build.